

Task 1: Install Snort and Verify Its Version

Objective

To install Snort on a system and verify that it is working.

Procedure

1. Open the terminal on your Snort-enabled system.
2. Run the following command:
 - a. `snort -V`
3. Observe the version information displayed by Snort.

Task 2: Create a Snort Rule to Detect the Word "admin"

Objective

To create a custom Snort rule that detects HTTP requests containing the word admin in the URL and generates an alert.

Procedure

Step 1: Open the local rules file.

On a typical Snort 2 installation:

- `sudo nano /etc/snort/rules/local.rules`

Step 2: Add the following rule.

- `alert tcp any any -> any 80 (msg:"HTTP URL contains admin"; flow:to_server,established; content:"admin"; http_uri; sid:1000001; rev:1;)`

Step 3: Save the file and test the configuration.

- `sudo snort -T -c /etc/snort/snort.conf`

Step 4: Generate a test request.

On your own lab web server, run:

- curl "<http://127.0.0.1/admin>"

Task 3: Simulate a SQL Injection Request

Objective

To generate a SQL injection test string against a local web application and observe whether Snort detects it.

Lab Setup

- Snort installed on the monitoring system.
- A local web server or deliberately vulnerable test application.
- A test endpoint that is owned by you or explicitly authorized for testing.

Step 1: Add a SQL injection detection rule

Open local.rules:

- sudo nano /etc/snort/rules/local.rules

Add this rule:

- alert tcp any any -> any 80 (msg:"Possible SQL Injection Attempt"; flow:to_server,established; content:"OR 1=1"; nocase; http_uri; sid:1000002; rev:1;)

Step 2: Send a crafted request to your local test application

For a test endpoint running on localhost:

```
curl --get \  
  --data-urlencode "id=' OR 1=1--" \  
  "http://127.0.0.1/test.php"
```

Step 3: Observe the Snort alert

Run Snort using your actual monitoring interface. For example, on Linux:

- `sudo snort -i eth0 -c /etc/snort/snort.conf -A console`

Task 4: Analyse the Snort Alert Logs

Objective

To examine the alerts produced by the SQL injection simulation and identify the traffic characteristics that triggered detection.

Sample Alert Analysis

The following is an illustrative example. Replace it with the actual alert generated in your lab.

Anomalous Traffic Patterns

1. SQL-related input: The HTTP request contained the expression `OR 1=1`, which is commonly used in SQL injection testing.
2. Unexpected query structure: The input included SQL operators and a comment marker that could alter the structure of a vulnerable SQL query.
3. Suspicious request parameter: The payload was supplied as a value in the HTTP request rather than as an ordinary identifier.
4. Rule match: Snort matched the configured content pattern and generated an alert.

Task 5: Detect Multiple Failed Login Attempts Using a Threshold Rule

Objective

To configure a Snort rule that detects repeated login attempts from the same source IP address within one minute.

1. Understanding Threshold-Based Detection

A threshold rule limits how often alerts are generated for matching traffic. It can help identify repeated login attempts, password guessing, and other potentially suspicious activity.

For this practical, we will monitor repeated requests to a login endpoint. This detects repeated login attempts, but it does not independently prove that each attempt failed.

2. Configure the Rule

Open the local rules file:

- `sudo nano /etc/snort/rules/local.rules`

Add the following Snort 2 rule:

- `alert tcp any any -> any 80 (msg:"Possible Brute Force - Repeated Login Attempts"; flow:to_server,established; content:"/login"; http_uri; detection_filter:track by_src, count 5, seconds 60; sid:1000003; rev:1;)`

3. Explanation

Component	Description
<code>content:"/login"</code>	Matches requests to the login endpoint
<code>http_uri</code>	Examines the HTTP URI
<code>track by_src</code>	Tracks matching requests by source IP
<code>count 5</code>	Sets the detection threshold to five matches
<code>seconds 60</code>	Uses a 60-second detection window
<code>sid:1000003</code>	Identifies the custom rule

4. Simulate Repeated Login Attempts

Use your own local web application. The following Python script sends six test requests to a local login endpoint:

```
import urllib.request
import urllib.error

url = "http://127.0.0.1/login"

for attempt in range(6):
    data = b"username=test&password=wrongpassword"
    request = urllib.request.Request(
        url,
        data=data,
        headers={
            "Content-Type": "application/x-www-form-urlencoded"
        }
    )

    try:
        with urllib.request.urlopen(request) as response:
            print(attempt + 1, response.status)
    except urllib.error.HTTPError as error:
        print(attempt + 1, error.code)
    except urllib.error.URLError as error:
        print("Connection error:", error.reason)
```

Run it using:

- `python3 login_test.py`

5. Verify the Alerts

Check the console or configured Snort alert log. Record the number of matching requests, the source IP address, the time window, and the alert message.

Security improvement: For accurate failed-login detection, correlate authentication failures recorded by the web application with network alerts. Repeated requests to /login alone cannot establish whether authentication succeeded or failed.